

我是通过**chatgpt**整理的, 但是这里基本都是原题, 我自己做了需要修改的部分

Mock Interview Question (Shipping Cost with Quantity)

Stripe often works with complex pricing and billing logic. Let's simulate that.

Problem Statement

You are asked to implement a shipping cost calculator for customer orders.

- Each **order** contains items with quantities, e.g., `{ "mouse": 20, "laptop": 5 }`.
 - Shipping cost depends on:
 - Country (**US**, **CA**)
 - Product type (**mouse**, **laptop**)
 - Quantity ordered
 - Pricing type: **fixed** or **incremental**
-

Part 1: Simple Fixed Pricing

In the simplest case, every product has a flat per-unit shipping cost, regardless of quantity.

- **Rules:**
 - US: mouse = \$550, laptop = \$1000
 - CA: mouse = \$750, laptop = \$1100

Example (US Order: 20 mice + 5 laptops):

$$20 \times 550 = 11,000$$

$$5 \times 1000 = 5,000$$

$$\text{Total} = 16,000$$

Part 2: Tiered Incremental Pricing

Costs may change based on quantity thresholds.

- **Rule for US laptops:**
 - First 2 units: \$1000 each
 - From the 3rd unit onward: \$900 each

Example (US Order: 5 laptops):

$$2 \times 1000 = 2,000$$

$$3 \times 900 = 2,700$$

$$\text{Total} = 4,700$$

Part 3: Mixed Pricing (Fixed + Incremental)

Some tiers may use *fixed* pricing: if you buy any quantity in that range, you pay the flat fee once.

- **Rule for US laptops:**
 - 0–2 units: fixed \$1000 (pay once, no matter if you buy 1 or 2)
 - 3+ units: incremental \$900 each

Example (US Order: 5 laptops):

Tier1: \$1000 (fixed, covers first 2 units)

$$\text{Tier2: } 3 \times 900 = 2,700$$

$$\text{Total} = 3,700$$

Your Task

Implement a function:

```
def calculate_shipping(order: dict, rules: dict, country: str) -> int:  
    # TODO: return total shipping cost
```

Given an order and shipping rules (see below), return the total shipping cost.

Example Input:

```
order = {"mouse": 20, "laptop": 5}
country = "US"
rules = {
  "US": {
    "mouse": [ { "type": "incremental", "min": 0, "max": float("inf"), "cost":
550 } ],
    "laptop": [
      { "type": "fixed", "min": 0, "max": 2, "cost": 1000 },
      { "type": "incremental", "min": 3, "max": float("inf"), "cost": 900 }
    ]
  },
  "CA": {
    "mouse": [ { "type": "incremental", "min": 0, "max": float("inf"), "cost":
750 } ],
    "laptop": [ { "type": "incremental", "min": 0, "max": float("inf"),
"cost": 1100 } ]
  }
}
```

Expected Output:

```
calculate_shipping(order, rules, "US")
# => 11,000 (mice) + 3,700 (laptops) = 14,700
```

Follow-up Questions interviewers may ask:

1. How would you extend your solution to handle new products or countries?
2. How do you handle edge cases (quantity = 0, invalid product, missing rule)?
3. Could your logic support more complex tiering (e.g., 0–5 fixed, 6–10 incremental, 11+ discounted)?
4. What's the runtime complexity of your solution?

Mock Interview Question (KAC Ring Simulation)

You are given a **KAC Ring** of n elements, initially all set to 0 except for one element set to 1 at position 0. At each step, the “1” moves clockwise around the ring. After a large number of steps, we want to know the distribution (ratio) of times each position contains a “1.”

Part 1: Simple Movement

- At each step, the “1” moves forward **one position**.
- After m steps, calculate the number of times each position has contained the “1.”

👉 This can be solved by brute force (simulate step by step), but may be too slow if n and m are large.

Part 2: Generalized Movement

- Instead of moving exactly 1 step, now the “1” moves forward **k steps** at a time (wrapping around).
- After m steps, calculate the distribution of visits for each position.

👉 This can be optimized with **prefix sums / modular arithmetic** to achieve $O(1)$ per query.

Input Format

n m k

- n = number of positions on the ring
 - m = number of steps
 - k = step size (if $k = 1$, it's the simple version)
-

Output Format

- Print an array of length n where `output[i]` = number of times position i contained the “1.”

Example 1

Input:

$n = 5, m = 7, k = 1$

Output:

[2, 2, 1, 1, 1]

Explanation:

- Start at index 0. Steps go: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 0 \rightarrow 1$.
- Counts: $\text{pos0}=2, \text{pos1}=2, \text{others}=1$.

Example 2

Input:

$n = 5, m = 6, k = 2$

Output:

[2, 0, 2, 0, 2]

Explanation:

- Steps: $0 \rightarrow 2 \rightarrow 4 \rightarrow 1 \rightarrow 3 \rightarrow 0 \rightarrow 2$
- Visits: $\text{pos0}=2, \text{pos2}=2, \text{pos4}=2, \text{pos1}=0, \text{pos3}=0$.

Constraints

- $1 \leq n \leq 10^5$
 - $1 \leq m \leq 10^{12}$ (large m means simulation must be optimized)
 - $1 \leq k \leq n$
-

Follow-up Questions

1. How would you handle **very large m** without simulating step by step?
 - Hint: detect cycles using `gcd(n, k)`.
2. Can you explain the **time complexity** of both brute force and optimized approaches?
3. How would you adapt your solution if we had **multiple starting positions** with “1”s instead of just one?

Mock Interview Question (ApplicationID Parsing - Part 1)

Credit and debit cards can support multiple payment networks (e.g., Visa, Mastercard, Girocard) and multiple applications (credit, debit, prepaid, etc.). Each payment method is stored on the card as an **ApplicationID** (AID), a unique identifier string.

Because transmitting ApplicationIDs between card and terminal has low bandwidth, the card compresses them into a single concatenated string:

- Each ApplicationID is prefixed by a **two-digit number** indicating its length.
 - ApplicationIDs are concatenated together.
 - The string ends with a trailing `0` to mark the end.
-

Example

Suppose a card supports these ApplicationIDs:

- `D27600002545500100`
- `A000000003101001`
- `A000000003101002`

The card encodes them into this string:

`"18D2760000254550010016A00000000310100116A0000000031010020"`

Breaking it down:

- `18` → next AID length = 18 → `D27600002545500100`
 - `16` → next AID length = 16 → `A000000003101001`
 - `16` → next AID length = 16 → `A000000003101002`
 - `0` → terminator
-

Your Task

Write a function:

```
def parse_application_ids(encoded: str) -> list[str]:  
    # TODO
```

that takes the encoded string and returns a **list of ApplicationIDs**, in the same order, without the length prefixes or trailing zero.

Example Input

```
encoded = "18D2760000254550010016A00000000310100116A0000000031010020"
```

Example Output

```
["D27600002545500100", "A000000003101001", "A000000003101002"]
```

Follow-up Questions (interviewer might ask)

1. How would you handle **invalid inputs** (e.g., wrong length, missing terminator)?
2. What's the **time complexity** of your solution?
3. How would you extend your solution if ApplicationIDs could be encoded in **binary form** instead of ASCII?

Mock Interview Question (ApplicationID Selection - Part 2 & 3)

We are building a payment terminal that supports multiple payment applications. When a customer inserts a credit/debit card, the card sends us its list of **ApplicationIDs** (AIDs) in the encoded format from Part 1.

Our terminal does not support every possible AID, but it has its own list of supported AIDs. We need to **select the intersection** between the card's ApplicationIDs and the terminal's supported ApplicationIDs, **preserving the order from the card**.

Part 2: Exact Matching

Your Task:

Write a function:

```
def filter_supported_applications(card_encoded: str, terminal_supported:
set[str]) -> list[str]:
    # TODO
```

- `card_encoded`: the encoded ApplicationID string (from Part 1)
- `terminal_supported`: a set/list of ApplicationIDs supported by the terminal

Return a **list of ApplicationIDs** that both the card and terminal support, preserving the order in which they appear on the card.

Example:

Card ApplicationIDs (encoded):

```
"18D2760000254550010016A00000000310100116A0000000031010020"
```

Terminal Supported ApplicationIDs:

```
["D27600002545500100", "A000000003101001"]
```

Output:

```
["D27600002545500100", "A000000003101001"]
```

Part 3: Partial Matching (RIDs)

An ApplicationID (AID) is made of:

- **RID** = first 10 chars → identifies the card scheme (e.g., Visa, Mastercard)
- **PIX** = rest of the string → identifies product/application

To save space, cards may store only an RID. Example:

"A000000003"

This means the card supports **all ApplicationIDs beginning with this RID** (all Visa AIDs).

Your Task (Extended):

Modify your function to support **partial matching**:

- If the card provides a full AID → match exact string.
- If the card provides only a RID → match all terminal-supported AIDs that begin with that RID.
- Avoid duplicates in the output.

Example:

Card ApplicationIDs (encoded):

"18D2760000254550010016A00000000310100110A0000000030"

Terminal Supported ApplicationIDs:

["A000000003101001", "A000000003101002", "D27600002545500100"]

Output:

["D27600002545500100", "A000000003101001", "A000000003101002"]

Follow-up Questions an Interviewer Might Ask

1. How would you handle **duplicates** if the RID expansion overlaps with explicitly listed AIDs?

2. What's the **time complexity** of your implementation? Could it scale to thousands of AIDs?
3. How would you modify your solution if the terminal also supported **wildcards** (like "A000000003*")?
4. How do you ensure **security** — i.e., making sure a malicious card doesn't send malformed AID strings?

Stripe Team Screen Mock Question

You have **45 minutes** to complete all 3 parts (plus 15 minutes buffer).

Part 1: Authorization Request Report

Stripe accepts payments on behalf of merchants. These payments begin as **Authorization Requests**, which represent an intent to transfer funds from an end user's credit card to a merchant.

Each Authorization Request arrives as a CSV line in the format:

```
timestamp_seconds,unique_id,amount,card_number,merchant
```

- `timestamp_seconds`: integer > 0 (in seconds).
- `unique_id`: request identifier.
- `amount`: decimal > 0.
- `card_number`: numeric string.
- `merchant`: alphanumeric with underscores or periods.

Policy (for now): All requests are **APPROVED**.

Your task:

- Write a function that parses the requests and produces a **human-readable report**.
- Each report line should contain:

```
timestamp unique_id amount outcome
```

separated by spaces, ordered chronologically.

Example Input:

```
5,R1,5.60,4242424242424242,bobs_burgers
10,R2,500.00,4242111111111111,a_corp
```

Expected Output:

```
5 R1 5.60 APPROVE
10 R2 500.00 APPROVE
```

You should also write **tests** to demonstrate correctness.

Part 2: Fraud Rules

A separate system maintained by Stripe's data scientists streams in **Fraud Rules**, which specify conditions that make certain requests fraudulent. Fraud rules are dynamic and once introduced, remain in effect forever.

Fraud rules arrive in this format:

```
time, field, value
```

- **time**: integer timestamp (seconds).
- **field**: which request field to check (**merchant**, **card_number**, etc.).
- **value**: the fraudulent value.

A rule means:

- From **time** onward, any Authorization Request where **field == value** must be **REJECTED**.
- Rules do **not** apply retroactively to earlier requests.

Example Rules:

```
1, merchant, bobs_burgers
20, card_number, 4242111111111111
```

Interpretation:

- At **t=1**, reject all future requests where **merchant == bobs_burgers**.
- At **t=20**, reject all future requests where **card_number == 4242111111111111**.

Example Requests:

```
5, R1, 5.60, 4242424242424242, bobs_burgers
10, R2, 500.00, 4242111111111111, a_corp
```

Expected Report:

```
5 R1 5.60 REJECT
10 R2 500.00 APPROVE
```

💡 Part 3 (not shown yet in your notes) would likely extend this system with **more fraud conditions or stateful checks** (e.g., spending limits, multiple rules per request, reversals, etc.).

Mock Stripe Question: Accounts, Users, and Roles

Stripe's system models organizations and workspaces as accounts. Accounts can form a hierarchy (an org contains workspaces, a workspace may contain child accounts, etc.). Users can be assigned different roles at different levels of the hierarchy.

You will implement functions to query this structure in multiple parts.

Input Data

1. **Accounts:**
A list of accounts with their parent relationship.

```
accounts = [
  {"accountId": "org_1", "parent": None},
  {"accountId": "wksp_1", "parent": "org_1"},
  {"accountId": "wksp_2", "parent": "org_1"},
  {"accountId": "proj_1", "parent": "wksp_1"}
]
```

- Each account has at most one parent.
- There are no cycles.

2. User Role Assignments:

A list of user → account → role assignments.

```
user_role_assignments = [  
    {"userId": "usr_1", "accountId": "org_1", "role": "admin"},  
    {"userId": "usr_2", "accountId": "wksp_1", "role": "viewer"},  
    {"userId": "usr_3", "accountId": "proj_1", "role": "editor"}  
]
```

Part 1 — Roles for a User at an Account

Write a function:

```
getRoles(userId: str, accountId: str) -> list[str]
```

Return all roles the given user has at that account.

Part 2 — Roles with Inheritance

Extend `getRoles`:

- Roles propagate **downward** in the hierarchy.
- A user with a role on a parent account also has that role on all its descendants.

So:

```
getRoles("usr_1", "wksp_1") # inherits "admin" from org_1
```

should return `["admin"]`.

Part 3 — Users at an Account

Write a function:

```
getUsers(accountId: str) -> list[str]
```

Return all users who have **any role** on the given account (directly or inherited).

Part 4 — Users with Role Filters

Modify `getUsers`:

```
getUsersWithFilter(accountId: str, rolesFilter: list[str]) -> list[str]
```

Return only the users who have **all the roles** in `rolesFilter` on the given account (direct or inherited).

Example:

```
print(getUsersWithFilter("wksp_1", []))
```

Expected output:

```
["usr_1", "usr_2", "usr_3"]
```

Constraints & Notes

- Assume there are $\leq 10^5$ accounts and users.
- Optimize for repeated queries (hint: precompute ancestors/descendants).
- Preserve output order as insertion order of users (if not otherwise specified).
-

Mock Interview Question: Currency Conversion System

You are given a set of **currency conversion rates** as a string input:

```
"AUD:USD:0.7, AUD:JPY:100, USD:CAD:1.2"
```

This means:

- 1 AUD = 0.7 USD
- 1 AUD = 100 JPY
- 1 USD = 1.2 CAD

You should parse the string into a data structure and then implement a conversion system with the following requirements.

Part 1 — Direct Conversion (warm-up)

Write a function:

```
def convert(from_currency: str, to_currency: str, amount: float) -> float:  
    ...
```

If a direct conversion rate exists, return the converted amount.

- Example: `convert("AUD", "USD", 10) → 7.0`
 - If no direct rate exists, return `None`.
-

Part 2 — Single-Hop Conversion

Now allow one intermediate step. If no direct rate exists, try converting through one other currency.

- Example: given AUD→USD and USD→CAD, you should be able to do AUD → CAD through USD.
- Input: `convert("AUD", "CAD", 10) → 8.4` (because 10 AUD → 7 USD → 8.4 CAD).

Part 3 — Multi-Hop + Best Rate

Now allow **multi-hop conversions** (chain through multiple currencies). Sometimes there are multiple paths, and you need to pick the **best possible conversion rate**.

- Example: with **AUD→GBP→CAD** and **AUD→USD→CAD**, you should try both paths and return whichever yields more CAD.
- Discussion: what does “best” mean? In this problem, assume best = **maximized final amount**.

Hint: Think of currencies as **graph nodes**, exchange rates as **edges**, and you need to run a **graph search (BFS/DFS)** or even a **max product path** problem.

Part 4 — Arbitrary Conversion Requests

Extend the system so that any conversion request can be answered dynamically, even if multiple steps are required.

Example Input:

AUD:USD:0.7, AUD:JPY:100, USD:CAD:1.2, JPY:CAD:0.011

-
- Query: **convert("AUD", "CAD", 10)**
 - Path 1: AUD→USD→CAD = $10 * 0.7 * 1.2 = 8.4$
 - Path 2: AUD→JPY→CAD = $10 * 100 * 0.011 = 11.0$
 - Return **11.0** because it's better.

♦ Follow-up Questions (interviewer style)

- How would you handle **cycles** (like AUD→USD, USD→AUD)?
- Could there be **arbitrage opportunities** (profitable loops)? How would you detect them?
- What's the **runtime complexity** of your solution? Could you handle 10k+ currencies?
- How would you scale this if exchange rates keep changing in real time?

Mock Interview: Business Company Dataset Verification

You are given a **dataset string** in CSV format.

The first row is the header:

`col1,col2,col3,col4,col5,col6`

-
- Following rows are the data to verify.

Your task: validate each row and print (or return) the results **in the original order**.

Verification Rules (case-insensitive)

1. **No empty columns** → if any column is empty, mark as `NOT VERIFIED`.
2. **col5 length rule** → must satisfy `5 <= len(col5) <= 31`.

Banned words in col2 → if col2 contains any of these words:

`COMPANY, FIRM, CORPORATION, LIMITED, LTD`

3. then mark as `NOT VERIFIED`. Case-insensitive (`company`, `CoMpAnY` are all invalid).
4. **Word overlap rule** →
 - Split `col2`, `col4`, and `col5` into word sets (split by spaces, ignore case, ignore punctuation).
 - Ignore stopwords `LLC` and `INC`.
 - At least **50% of words from col2** must overlap with words in **either col4 OR col5**.

- If col2 has no valid words after filtering, treat as NOT VERIFIED.

If all rules pass → row is VERIFIED. Otherwise → NOT VERIFIED.

◆ Output Format

Version A (basic)

Print two sections:

VERIFIED: <comma+space joined col2 values>

NOT VERIFIED: <comma+space joined col2 values>

Version B (with reasons)

For each failed row, print:

<col2> is not verified because <reason>

Multiple reasons can be combined with semicolons.

Example 1 (basic)

Input

```
col1,col2,col3,col4,col5,col6
a,"green valley llc",c,"green valley park","green valley project",f
x,"north firm",y,"north hub","north hub",z
m,"alpha beta",n,"gamma","alpha site",p
```

Explanation

- Row 1: passes all rules ✓
- Row 2: col2 contains banned word firm ✗
- Row 3: col2 {alpha, beta}, col5 {alpha, site}, overlap=1/2=50% ✓

Output

VERIFIED: green valley llc, alpha beta
NOT VERIFIED: north firm

Example 2 (with reasons)

Input

```
col1,col2,col3,col4,col5,col6  
a,"west corp",c,"west lake","we",f  
x,"north firm",y,"","north hub",z
```

Explanation

- Row 1: col5 length = 2 (too short) ❌
- Row 2: col4 is empty ❌, col2 has banned word `firm` ❌

Output

```
west corp is not verified because col5 length not in [5,31]  
north firm is not verified because col4 is empty; col2 contains a banned word
```

◆ Follow-up Questions (Stripe-style)

- How would you handle **large input datasets** efficiently?
- How to make the **rules configurable** (e.g., banned words or length range)?
- How do you ensure **unit tests** cover edge cases:
 - Empty column,
 - Exactly length 5 or 31,
 - col2 = only "LLC",
 - Overlap exactly 50% vs <50%.

Mock Interview: Payment & Invoice Reconciliation

You are given a list of **invoices** and a single **payment**. Implement a reconciliation function in three incremental parts.

Data model

- **Invoice:** (`invoice_id`: string, `amount`: number, `date`: YYYY-MM-DD)
 - Example: (`"invoice-id-1"`, `100`, `"2022-01-01"`)
- **Payment:** one object per test
 - Part 1 format: `{"payment_id": "pmt_1", "amount": 300, "paying_for": "invoice-id-1"}`
 - Part 2/3 format: `{"payment_id": "pmt_2", "amount": 300}` (no invoice id)

Output format

Print (or return) a sentence:

`<payment_id> paid <amount> amount for invoice <invoice_id> on date <invoice_date>`

For Part 3, if forgiveness is applied, append:

`(forgiven <forgiveness_amount>)`

If nothing matches (only applies in Parts 2–3), return/print:

`<payment_id> has no matching invoice`

Part 1 — Explicit invoice id

Input:

- A list of invoices
- A payment **that includes** `paying_for: <invoice_id>`

Rules:

- If `paying_for` matches an invoice id, produce the output line using that invoice's amount & date (ignore payment.amount mismatches for this part).
- Assume ids are valid in this part.

Example

Invoices:

```
("invoice-id-1", 100, "2022-01-01")  
("invoice-id-2", 300, "2022-01-30")
```

Payment:

```
{payment_id: "p_100", amount: 300, paying_for: "invoice-id-1"}
```

Output:

```
p_100 paid 300 amount for invoice invoice-id-1 on date 2022-01-01
```

Part 2 — Match by amount (no invoice id)

Input:

- Same invoices list
- A payment **without** `paying_for`

Rules:

- Find invoice(s) whose **invoice.amount == payment.amount**.
- If multiple match, choose the **earliest date** (tie-break by date ascending).
- Produce the same output line.
- If no invoice amount equals the payment amount, print the “no matching invoice” message.

Example

Invoices:

```
("invoice-id-1", 100, "2022-01-01")
("invoice-id-2", 300, "2022-01-15")
("invoice-id-3", 300, "2022-02-01")
```

Payment:

```
{payment_id: "p_200", amount: 300}
```

Output:

```
p_200 paid 300 amount for invoice invoice-id-2 on date 2022-01-15
```

Part 3 — Amount tolerance (forgiveness)

Input:

- Same invoices list
- A payment without `paying_for`
- A numeric **forgiveness** value $f \geq 0$

Rules:

- First try exact amount match (Part 2).
- If none found, allow matching any invoice where $\text{abs}(\text{invoice.amount} - \text{payment.amount}) \leq f$.
- If multiple candidates, still choose **earliest date**.
- Output the same sentence and append `"(forgiven <difference>)"` where $\text{difference} = \text{round}(\text{abs}(\text{invoice.amount} - \text{payment.amount}), 2)$ (or the format your language uses).
- If no invoice within tolerance, print "no matching invoice".

Example A (uses tolerance)

Invoices:


```
("invoice-id-10", 100, "2022-01-01")
("invoice-id-11", 295, "2022-01-05")
```

Payment:

```
{payment_id: "p_300", amount: 300}
forgiveness f = 5
```

Output:

```
p_300 paid 300 amount for invoice invoice-id-11 on date 2022-01-05 (forgiven
5)
```

Example B (exact beats tolerance)

Invoices:

```
("a", 300, "2022-01-03")
("b", 299, "2022-01-01")
```

Payment:

```
{payment_id: "p_301", amount: 300}
forgiveness f = 2
```

Output:

```
p_301 paid 300 amount for invoice a on date 2022-01-03
```

Notes & expected interview discussion

- **Priority order:**
 1. If `paying_for` is present → use that invoice id (Part 1).
 2. Otherwise try **exact amount** (Part 2).
 3. Otherwise try **tolerance** (Part 3).
- **Determinism:** when multiple candidates exist, pick **earliest invoice date**.
- **Parsing:** inputs may arrive as a single string; you should split by `\n` and `,`, trim quotes, and cast numbers safely.
- **Edge cases to test:** empty invoices list; multiple exact matches (date tie-break); amount as integer vs float; tolerance 0; negative or malformed amounts (reject/assume valid per prompt).

- **Complexity:** linear scan per payment is fine for small testcases; for many payments, suggest indexing by amount (e.g., hashmap amount→min-date invoice), and a secondary structure for tolerance (e.g., balanced tree or bucket by rounded amount).

Mock Interview: Fraud Detection

Stripe processes billions of transactions. Your job is to flag **merchants (accounts)** as fraudulent when too many of their transactions are suspicious.

Input is provided as a multi-line string. You must parse it and output a **lexicographically sorted, comma-separated list of fraudulent `account_ids`**.

Shared Input Sections (in order)

1. **Non-fraudulent codes** (comma separated)
2. **Fraudulent codes** (comma separated)
3. **MCC thresholds** — lines of: `MCC, threshold`
4. **Merchant MCCs** — lines of: `account_id, MCC`
5. **Minimum number of total transactions** before evaluating a merchant (integer ≥ 0)
6. **Events** — one per line:
 - Charge: `CHARGE, charge_id, account_id, amount, code`
 - (Part 3) Dispute: `DISPUTE, charge_id` → that charge no longer counts as fraudulent

You may assume all IDs/codes are strings without commas; amounts are positive numbers.

Part 1 — Count-Based Fraud Detection

Rules

- Each MCC (Merchant Category Code) has an **integer threshold T** ($T \geq 1$).
- For each account:
 - Count **fraudulent** charges = charges whose `code` is in the fraudulent list.
 - Count **total** charges (all codes, for Part 1 the total count is only needed for Part 2/3; here it's not used).

- If the merchant has **at least T fraudulent charges** for their MCC **and** the merchant has at least `minTransactions` total charges, mark the merchant as **fraudulent**.

Output: lexicographically sorted `account_ids`, comma-separated.

Example (Part 1)

Input

```
approved,invalid_pin,expired_card
do_not_honor,stolen_card,lost_card
retail,5
airline,2
restaurant,10
venue,3
acct_1,airline
acct_2,venue
acct_3,retail
0
CHARGE,ch_1,acct_1,100,do_not_honor
CHARGE,ch_2,acct_1,200,approved
CHARGE,ch_3,acct_1,300,do_not_honor
CHARGE,ch_4,acct_2,100,lost_card
CHARGE,ch_5,acct_2,200,lost_card
CHARGE,ch_6,acct_2,300,lost_card
CHARGE,ch_7,acct_3,100,lost_card
CHARGE,ch_8,acct_2,200,stolen_card
CHARGE,ch_9,acct_3,100,approved
```

Output

```
acct_1,acct_2
```

Why

- `acct_1` (airline, T=2): has 2 fraudulent charges → flagged.
 - `acct_2` (venue, T=3): has 4 fraudulent charges → flagged.
 - `acct_3` (retail, T=5): has 1 fraudulent → not flagged.
-

Part 2 — Percentage-Based Fraud Detection

The business changes thresholds to **fraud percentage** per MCC.

Rules

- MCC thresholds are **floats in [0,1]** (e.g., `retail,0.8` means 80%).
- A merchant is fraudulent if:
 - They have **at least** `minTransactions total charges`, and
 - `fraudulent_count / total_count >= threshold_for_merchant_MCC`.

The rest of the input format is unchanged (no disputes yet).

(You can reuse the Part 1 parser; only the threshold interpretation changes based on whether it parses as int vs float.)

Part 3 — Dispute Resolution

Sometimes transactions are incorrectly marked as fraudulent. We now support **disputes**:

New Event

`DISPUTE,charge_id`

When a dispute arrives, that charge should be treated as **not fraudulent for all calculations** (i.e., remove it from the merchant's fraud count, but it still counts toward total only if that's how your interviewer wants it—most versions remove it from fraud count and keep it as a normal/non-fraudulent charge).

If a merchant was above the threshold only because of **disputed** transactions, they may return to **non-fraudulent** status until they cross the threshold again.

Example (Part 3)

Input

```
approved,invalid_pin,expired_card
do_not_honor,stolen_card,lost_card
retail,0.8
venue,0.25
acct_1,retail
```

```
acct_2, retail
2
CHARGE, ch_1, acct_1, 100, do_not_honor
CHARGE, ch_2, acct_1, 200, lost_card
CHARGE, ch_3, acct_1, 300, do_not_honor
DISPUTE, ch_2
CHARGE, ch_4, acct_2, 400, lost_card
CHARGE, ch_5, acct_2, 500, lost_card
CHARGE, ch_6, acct_2, 600, lost_card
CHARGE, ch_7, acct_2, 700, lost_card
CHARGE, ch_8, acct_2, 800, do_not_honor
```

Output

```
acct_2
```

Why

- **acct_1** (retail, 0.8): originally had 3 fraud out of 3 (100%) → but **ch_2** disputed → now $2/3 \approx 0.667 < 0.8$ → not fraudulent.
- **acct_2** (retail, 0.8): has 5 fraud out of 5 (100%) → fraudulent.

Clarifications to ask (good interview hygiene)

- Should **disputed charges** still count toward **total**? (State your assumption; common answer: **yes**, count as non-fraudulent.)
 - If an MCC threshold line appears twice, which wins? (Assume unique.)
 - What to do with **codes not in either list**? (Treat them as non-fraudulent; or reject input—confirm.)
 - Is **minTransactions** applied per merchant as an absolute count? (**Yes.**)
 - Are there performance constraints? (Assume $O(\#events)$ one pass is fine, but mention stream processing.)
-

What interviewers look for

- **Parsing cleanly** and building small, composable helpers (parse codes, parse MCCs, parse charges, parse disputes).
- Correct **counting per account** and **lookup by MCC**.
- Handling **threshold as int (count) vs float (percentage)**.
- Properly applying **minTransactions**.
- Correctly **undoing** fraud for **DISPUTED** charges.
- **Deterministic output**: sorted account IDs, comma-separated.
- A couple of **unit tests** (happy path, minTransactions gate, boundary equals, disputes flipping status).

Subscription Notification Scheduler

Problem Description

You are asked to implement a notification system for subscription plans.

Each user subscribes to a plan with a start date and a duration (in days).

You must schedule and print emails at the correct dates based on the rules provided.

Part 1 – Basic Email Scheduling

Input

- **send_schedule** – A mapping from relative day offsets to email message types:
 - "start" → send on the subscription start day
 - Negative integers (e.g., -15) → send *before* subscription end date
 - "end" → send on the subscription end date
- **user_accounts** – A list of user dictionaries with:
 - name : string – user name
 - plan : string – subscription plan
 - account_date : int – day subscription started
 - duration : int – duration of subscription in days

Output

Print one line for each email event in ascending order of day.

Each line format:

php-template

複製程式碼

```
<day>: [<Email Type>] Subscription for <name> (<plan>)
```

Example

Input:

python

複製程式碼

```
send_schedule = {
    "start": "Welcome",
    -15: "Upcoming expiry",
    "end": "Expired"
}

user_accounts = [
    {"account_date": 0, "duration": 30, "name": "John", "plan": "Silver"},
    {"account_date": 1, "duration": 15, "name": "Alice", "plan": "Gold"}
]
```

Output:

graphql

複製程式碼

```
0: [Welcome] Subscription for John (Silver)
1: [Welcome] Subscription for Alice (Gold)
1: [Upcoming expiry] Subscription for Alice (Gold)
15: [Upcoming expiry] Subscription for John (Silver)
16: [Expired] Subscription for Alice (Gold)
30: [Expired] Subscription for John (Silver)
```



Part 2 – Handling Plan Changes

Now you are given an additional input: a list of plan change events.

When a user changes plan, you must:

- Print a `[Changed]` message on the change date
- Recalculate the remaining duration relative to the change date
- Schedule future notifications (upcoming expiry, expired) based on the new plan and updated timeline

Input

- **users** – same as Part 1
- **changes** – list of dictionaries:
 - `name`: user name
 - `new_plan`: new plan name
 - `change_date`: day the plan changed

Output

Include `[Changed]` events in chronological order, and base upcoming-expiry / expired events on the updated plan.

Example

Input:

python

複製程式碼

```
users = [  
    {"name": "A", "plan": "X", "begin_date": 0, "duration": 30},  
    {"name": "B", "plan": "Y", "begin_date": 1, "duration": 15}  
]  
  
changes = [  
    {"name": "A", "new_plan": "Y", "change_date": 5}  
]
```

Output:

makefile

複製程式碼

```
0: [Welcome] A, subscribe in plan X  
1: [Welcome] B, subscribe in plan Y  
1: [Upcoming expiration] B, subscribe in plan Y  
5: [Changed] A, subscribe in plan Y  
15: [Upcoming expiration] A, subscribe in plan Y  
16: [Expired] B, subscribe in plan Y  
30: [Expired] A, subscribe in plan Y
```



Part 3 – Bonus: Renewals

In addition to plan changes, you now have **renewal events** in the change list. Each renewal extends the subscription duration by the given number of days.

You must:

- Print `[Renewed]` on the renewal date
- Reschedule future emails based on the new end date

Input

- **changes** may now have two kinds of events:
 - Plan change: `{name, new_plan, change_date}`
 - Renewal: `{name, extension, change_date}`

Output

Include `[Renewed]` events and re-schedule upcoming-expiration and expired events accordingly.

Example

Input:

```
python

users = [
    {"name": "A", "plan": "X", "begin_date": 0, "duration": 30},
    {"name": "B", "plan": "Y", "begin_date": 1, "duration": 15}
]

changes = [
    {"name": "A", "new_plan": "Y", "change_date": 5},
    {"name": "B", "extension": 15, "change_date": 3}
]
```

複製程式碼

Output:

```
makefile

0: [Welcome] A, subscribe in plan X
1: [Welcome] B, subscribe in plan Y
1: [Upcoming expiration] B, subscribe in plan Y
3: [Renewed] B, subscribe in plan Y
5: [Changed] A, subscribe in plan Y
15: [Upcoming expiration] A, subscribe in plan Y
16: [Upcoming expiration] B, subscribe in plan Y
30: [Expired] A, subscribe in plan Y
30: [Expired] B, subscribe in plan Y
```

複製程式碼



—

Accept-Language Header Parser

Problem Description

You are building an HTTP server that needs to select an appropriate language for each client request based on the `Accept-Language` header.

The `Accept-Language` header is a comma-separated list of language tags, optionally including region subtags, describing the client's language preferences in order of priority (first = most preferred).

Your task is to write a function:

```
python 📄 複製程式碼  
  
def parse_accept_language(header: str, supported: list[str]) -> list[str]:  
    ...
```

that returns the subset of `supported` languages that are acceptable according to the client's `Accept-Language` header, in **descending order of preference**.

Part 1 – Exact Match

Match only exact language tags that appear in both the client's `Accept-Language` header and the server's supported language list.

Examples

```
python 📄 複製程式碼  
  
parse_accept_language(  
    "en-US, fr-CA, fr-FR",  
    ["fr-FR", "en-US"]  
)  
# returns ["en-US", "fr-FR"]  
  
parse_accept_language(  
    "fr-CA, fr-FR",  
    ["en-US", "fr-FR"]  
)  
# returns ["fr-FR"]  
  
parse_accept_language(  
    "en-US",  
    ["en-US", "fr-CA"]  
)  
# returns ["en-US"]
```

